

Group Assignment 2 (Group 10)

Student 1: Sri Satya Satwik Sirigineedi

Student 2: Harsh Deval Shah

Student 3: Puneeth Shankar

Student 4: Samar Kale

Note: The following implementations are based on the codes in the GitHub repository below

https://github.com/hds3304/RBE500_Group_Assessment/tree/assessment2/src

1 Objective

The goal of this assignment is to implement and test a velocity-based control pipeline for the OpenManipulator robot. The pipeline consists of three main components:

1) A **velocity kinematics node** that provides two services:

- Forward velocity kinematics: joint velocities $\dot{\mathbf{q}} \rightarrow$ end-effector linear velocities $\mathbf{v}$.
- Inverse velocity kinematics: end-effector linear velocities $\mathbf{v} \rightarrow$ joint velocities $\dot{\mathbf{q}}$.

2) An **incremental position reference node** that integrates joint velocities into joint position commands:

$$\mathbf{q}_{\text{ref}}(k+1) = \mathbf{q}_{\text{ref}}(k) + \dot{\mathbf{q}}(k) \Delta t.$$

3) A **linear motion task node** that commands a constant end-effector velocity in the positive y -direction in Cartesian space, converts it into joint velocities using the Jacobian, and feeds it into the incremental reference node so that the robot end-effector follows a straight line.

In addition, we record the Cartesian end-effector position over time and verify that the trajectory in task space is approximately linear in the $+y$ direction.

2 Velocity-Level Kinematics Node

2.1 Robot Geometry and Forward Kinematics

We base the kinematic model on the same geometry used in Part 1 (position-level forward kinematics). The link parameters are:

- $L_0 = 36.076$ mm (base offset in z),
- $L_1 = 60.25$ mm,
- $L_{2z} = 128.0$ mm, $L_{2x} = 24.0$ mm,
- $L_3 = 124.0$ mm,
- $L_4 = 133.4$ mm (tool offset).

Given the joint angles $\mathbf{q} = [q_1, q_2, q_3, q_4]^T$, we construct the homogeneous transformations

$$T_0^1(q_1), \quad T_1^2(q_2), \quad T_2^3(q_3), \quad T_3^4(q_4), \quad T_4^{ee},$$

and obtain the full transformation

$$T_0^{ee}(\mathbf{q}) = T_0^1 T_1^2 T_2^3 T_3^4 T_4^{ee}.$$

The end-effector position in the base frame is then

$$\mathbf{p}(\mathbf{q}) = \begin{bmatrix} x(\mathbf{q}) \\ y(\mathbf{q}) \\ z(\mathbf{q}) \end{bmatrix} = T_0^{ee}(\mathbf{q})_{1:3,4}.$$

This forward kinematics function is implemented in the node as a helper method `forward_kinematics(q)` returning the 3×1 position vector in millimeters.

2.2 Numerical Jacobian Computation

To obtain the velocity relationship between joint space and task space, we compute the 3×4 geometric Jacobian numerically using central differences. For each joint q_i we perturb by a small value ε :

$$\mathbf{q}_i^+ = \mathbf{q} + \varepsilon \mathbf{e}_i, \quad \mathbf{q}_i^- = \mathbf{q} - \varepsilon \mathbf{e}_i,$$

where $\mathbf{e}_i$ is the i -th basis vector. We then approximate each column of the Jacobian as:

$$\mathbf{J}_{\cdot i}(\mathbf{q}) \approx \frac{\mathbf{p}(\mathbf{q}_i^+) - \mathbf{p}(\mathbf{q}_i^-)}{2\varepsilon}.$$

This yields a numerical Jacobian $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{3 \times 4}$ such that

$$\mathbf{v} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}},$$

where $\dot{\mathbf{q}}$ are the joint velocities.

2.3 Forward Velocity Kinematics Service

The **forward velocity** service (`fk_velocity`) implements the mapping

$$\dot{\mathbf{q}} \rightarrow \mathbf{v}.$$

- The node subscribes to `/joint_states` to maintain the current joint configuration $\mathbf{q}_{\text{current}}$.
- The service request provides joint velocities $\dot{\mathbf{q}}$:

$$\text{request.joint_velocities} = [\dot{q}_1, \dot{q}_2, \dot{q}_3, \dot{q}_4].$$

- The node computes the Jacobian at the current configuration:

$$\mathbf{J} = \mathbf{J}(\mathbf{q}_{\text{current}}),$$

and evaluates

$$\mathbf{v} = \mathbf{J} \dot{\mathbf{q}}.$$

- The resulting Cartesian velocity is returned in the response:

$$\text{response.end_effector_velocities} = [\dot{x}, \dot{y}, \dot{z}].$$

The node also prints the input and output to the terminal for debugging:

$$\text{FK REQUEST: } \dot{\mathbf{q}} \mapsto \mathbf{v}.$$

2.4 Inverse Velocity Kinematics Service

The **inverse velocity** service (`ik_velocity`) implements the mapping

$$\mathbf{v} \rightarrow \dot{\mathbf{q}}.$$

- The request provides a desired Cartesian linear velocity:

$$\text{request.end_effector_velocities} = [\dot{x}, \dot{y}, \dot{z}].$$

- Using the same numerically computed Jacobian $\mathbf{J}(\mathbf{q}_{\text{current}})$, we compute its Moore–Penrose pseudoinverse $\mathbf{J}^+$:

$$\mathbf{J}^+ = \mathbf{J}^+(\mathbf{q}_{\text{current}}) = \text{pinv}(\mathbf{J}).$$

- The joint velocities are then obtained as:

$$\dot{\mathbf{q}} = \mathbf{J}^+ \mathbf{v}.$$

- These joint velocities are returned in the response:

$$\text{response.joint_velocities} = [\dot{q}_1, \dot{q}_2, \dot{q}_3, \dot{q}_4].$$

This service is later used by the linear motion task node to convert a constant $+y$ -direction Cartesian velocity into joint-space velocities.

3 Incremental Position Reference Node

3.1 Concept

The second node implements an **incremental joint-space integrator** that behaves like a simple velocity controller. Instead of directly commanding joint velocities to the robot, we integrate them into position references and send those to the joint position controllers.

Let Δt be the sampling time, and let $\dot{\mathbf{q}}(k)$ denote the commanded joint velocities at discrete time step k . The node maintains an internal reference vector $\mathbf{q}_{\text{ref}}(k)$ and updates it via:

$$\mathbf{q}_{\text{ref}}(k+1) = \mathbf{q}_{\text{ref}}(k) + \dot{\mathbf{q}}(k) \Delta t. \quad (1)$$

The updated reference $\mathbf{q}_{\text{ref}}(k+1)$ is then sent as a joint-space goal to the robot's `/goal_joint_space_path` controller.

3.2 Implementation Details

- **Initialization:** The node subscribes to `/joint_states`. On the first received message, it initializes

$$\mathbf{q}_{\text{ref}}(0) = \mathbf{q}_{\text{measured}}(0),$$

using the first four joint positions. A flag is set to indicate that the node is initialized.

- **Velocity Input:** The node subscribes to a topic

$$\text{/joint_velocity_cmd} : \dot{\mathbf{q}} \in \mathbb{R}^4,$$

which provides the desired joint velocities at each time step.

- **Integration Step:** When a new velocity command $\dot{\mathbf{q}}(k)$ is received and the node is initialized, the internal reference is updated as:

$$\mathbf{q}_{\text{ref}}(k+1) = \mathbf{q}_{\text{ref}}(k) + \dot{\mathbf{q}}(k) \Delta t,$$

where we chose $\Delta t = 0.1$ s (10 Hz sampling).

- **Sending Joint Goals:** The updated reference is sent via the `SetJointPosition` service to the OpenManipulator controller:

- `planning_group` = "arm",
- joint names: ["joint1", "joint2", "joint3", "joint4", "gripper"],
- joint positions: $\mathbf{q}_{\text{ref}}$ for the arm joints, and 0.0 for the gripper,
- `path_time` is set equal to Δt , so that each reference is executed over one sampling interval.

This design effectively makes the incremental node behave like a velocity controller, but implemented through streaming position setpoints to the low-level joint position controller.

```

root@rez:/workspaces/RBE_500/colcon_ws 10x30
root@rez:/workspaces/RBE_500/colcon_ws# ros2 run ga2_velocity_kinematics
[INFO] [1764900163.302921826] [velocity_kinematics_node]: Velocity Kinematics Node Ready.
[INFO] [1764900169.578921137] [velocity_kinematics_node]: > FK REQUEST: Joints Vel [-0.00055352 0.0 0.003533 -0.00338322 -0.00173135] -> EE Vel [ 0.49884745 -0.0012103 -0.00097081]
[INFO] [1764900174.973336636] [velocity_kinematics_node]: > FK REQUEST: Joints Vel [ 0.00156972 0.0 0.0125287 -0.001193 -0.00061051] -> EE Vel [-1.23593126e-03 5.01137703e-01 -3.14169842e-04]
[INFO] [1764900181.402610577] [velocity_kinematics_node]: > FK REQUEST: Joints Vel [ 0. -0.0009 -0.0006 -0.0003] -> EE Vel [-0.00464505 -0.00163795 0.45986184]

root@rez:/workspaces/RBE_500/colcon_ws 10x30
root@rez:/workspaces/RBE_500/colcon_ws# ros2 service call /fk_velocity ga2_srv/srv/ForwardVelocity jo
int_velocities: [-0.00055352, 0.003533, -0.00338322, -0.00173135]
requester: making request: ga2_srv.srv.ForwardVelocity_Request(joint_velocities=[-0.00055352, 0.003533, -0.00338322, -0.00173135])
response:
ga2_srv.srv.ForwardVelocity_Response(end_effector_velocities=[0.49884745114360993, -0.0012103027530343709, -0.0009708099851229168])
root@rez:/workspaces/RBE_500/colcon_ws# ros2 service call /fk_velocity ga2_srv/srv/ForwardVelocity jo
int_velocities: [0.00156972, 0.00125287, -0.001193, -0.00061051]
requester: making request: ga2_srv.srv.ForwardVelocity_Request(joint_velocities=[0.00156972, 0.00125287, -0.001193, -0.00061051])
response:
ga2_srv.srv.ForwardVelocity_Response(end_effector_velocities=[-0.0012359312643691958, 0.5011377030126245, -0.0003141698422366447])
root@rez:/workspaces/RBE_500/colcon_ws# ros2 service call /fk_velocity ga2_srv/srv/ForwardVelocity jo
int_velocities: [0.0, -0.0009, -0.0006, -0.0003]
requester: making request: ga2_srv.srv.ForwardVelocity_Request(joint_velocities=[0.0, -0.0009, -0.0006, -0.0003])
response:
ga2_srv.srv.ForwardVelocity_Response(end_effector_velocities=[-0.00464505253520997, -0.00163795292377953, 0.459861836460645])
root@rez:/workspaces/RBE_500/colcon_ws#

```

Figure 1: Terminal output from the `fk_velocity` service, showing joint velocity inputs and the corresponding end-effector linear velocities.

```

root@rez:/workspaces/RBE_500/colcon_ws 10x30
root@rez:/workspaces/RBE_500/colcon_ws# ros2 run ga2_velocity_kinematics
[INFO] [1764899506.615515692] [velocity_kinematics_node]: Velocity Kinematics Node Ready.
[INFO] [1764899515.446028011] [velocity_kinematics_node]: > IK REQUEST: EE Vel [0.5 0. 0.] -> Joints Vel [-0.00055352 0.003533 -0.00338322 -0.00173135]
[INFO] [1764899524.067732409] [velocity_kinematics_node]: > IK REQUEST: EE Vel [0. 0.5 0.] -> Joints Vel [0.00156972 0.00125287 -0.001193 -0.00061051]
[INFO] [1764899532.179408647] [velocity_kinematics_node]: > IK REQUEST: EE Vel [0. 0. 0.5] -> Joints Vel [-5.51937597e-10 -9.65346783e-04 -0.63616263e-04 -3.44694343e-04]

root@rez:/workspaces/RBE_500/colcon_ws 10x30
root@rez:/workspaces/RBE_500/colcon_ws# ros2 service call /ik_velocity ga2_srv/srv/InverseVelocity en
d_effector_velocities: [0.5,0,0]
requester: making request: ga2_srv.srv.InverseVelocity_Request(end_effector_velocities=[0.5, 0.0, 0.0])
response:
ga2_srv.srv.InverseVelocity_Response(joint_velocities=[-0.0005535210395920703, 0.0035330040022555, -0.003383219659890188, -0.0017313480480547047])
root@rez:/workspaces/RBE_500/colcon_ws# ros2 service call /ik_velocity ga2_srv/srv/InverseVelocity en
d_effector_velocities: [0,0.5,0]
requester: making request: ga2_srv.srv.InverseVelocity_Request(end_effector_velocities=[0.0, 0.5, 0.0])
response:
ga2_srv.srv.InverseVelocity_Response(joint_velocities=[0.0015697241789055542, 0.0012528713613727655, -0.0011930014766150083, -0.0006105133528133192])
root@rez:/workspaces/RBE_500/colcon_ws# ros2 service call /ik_velocity ga2_srv/srv/InverseVelocity en
d_effector_velocities: [0,0,0.5]
requester: making request: ga2_srv.srv.InverseVelocity_Request(end_effector_velocities=[0.0, 0.0, 0.5])
response:
ga2_srv.srv.InverseVelocity_Response(joint_velocities=[-5.519375971154269e-10, -0.0009653467826819777, -0.00063616262661653, -0.0003446943427559229])
root@rez:/workspaces/RBE_500/colcon_ws#

```

Figure 2: Terminal output from the `ik_velocity` service, converting a Cartesian velocity request into joint velocities using the Jacobian pseudoinverse.

4 Linear Motion in the Positive y -Direction

4.1 Task Description

The third part of the assignment asks us to move the end-effector along a straight line in the positive y -direction of the base frame at a constant Cartesian velocity. The desired behavior is:

$$\mathbf{v}_{\text{desired}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ v_y \\ 0 \end{bmatrix},$$

with a constant $v_y > 0$, and $\dot{x} = \dot{z} = 0$. This Cartesian velocity is converted to joint space using the inverse velocity service, and the resulting joint velocities are fed into the incremental position reference node.

4.2 Linear Motion Task Node

The `linear_motion` node implements this experiment:

- We choose a constant velocity in the $+y$ direction:

$$v_x = 0, \quad v_y = 20 \text{ mm/s}, \quad v_z = 0,$$

which is consistent with the FK units (millimeters).

- The experiment is run for a duration of $T = 5$ s at a rate of 10 Hz, giving $N = 50$ steps. At each step k :
 1. A request is sent to the `ik_velocity` service with the desired Cartesian velocity $\mathbf{v}_{\text{desired}}$.
 2. The service computes joint velocities:

$$\dot{\mathbf{q}}(k) = \mathbf{J}^+(\mathbf{q}_{\text{current}}) \mathbf{v}_{\text{desired}}.$$

3. These joint velocities are published on the topic `/joint_velocity_cmd`.
4. The incremental node integrates $\dot{\mathbf{q}}(k)$ into $\mathbf{q}_{\text{ref}}(k+1)$ and sends the new joint-space goal to the robot.

- At the end of the experiment, a zero-velocity command ($\dot{\mathbf{q}} = \mathbf{0}$) is sent to bring the motion to a stop.

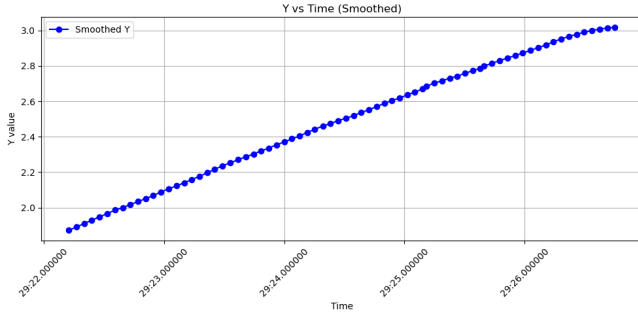
4.3 Recording and Plotting Cartesian Trajectory

To verify that the end-effector moves along a straight line, we record the Cartesian position over time:

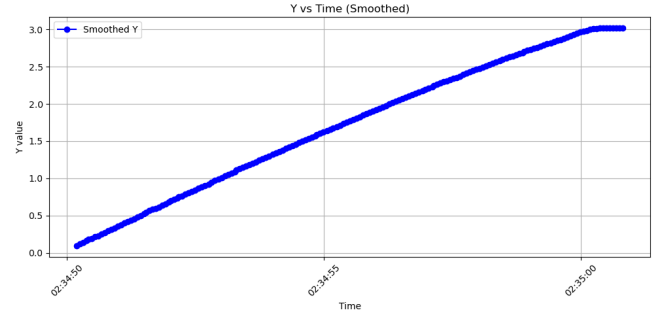
- The velocity kinematics node continuously receives `/joint_states` and can compute the corresponding end-effector position $\mathbf{p}(t)$ using the forward kinematics function.
- This position is published on a topic (e.g., `/ee_position`) as a 3-element vector $[x(t), y(t), z(t)]^T$ at 10 Hz.
- During the experiment, we log this data and plot it using matplotlib.

We expect the y-position to increase approximately linearly over time, reflecting the constant positive v_y .

Example plots of the trajectory in the y -direction over time is shown in Figure 3, demonstrating that the end-effector follows a nearly straight line in the $+y$ direction.



(a) Plot for 5s



(b) Plot for 10s

Figure 3: Recorded end-effector y-position vs. time during the constant-velocity linear motion experiment for 5s and 10s respectively.

5 Conclusion

In this assignment we implemented a complete velocity-based control pipeline for the OpenManipulator:

- A **velocity kinematics node** providing forward and inverse velocity mappings using a numerically computed Jacobian.
- An **incremental position reference node** that integrates joint velocities into reference positions and streams them to the robot's joint position controller.
- A **linear motion task node** that generates a constant end-effector velocity in the positive y -direction, converts it to joint velocities, and drives the manipulator along an approximately straight line in task space.

The recorded Cartesian trajectory confirms that the end-effector motion in the y -direction is close to linear, validating both the Jacobian-based velocity mappings and the incremental position control strategy.

6 Run-Time Images

- Initial Robot Position



- Run-Time Images



- Final Robot Position

